

Bab 4 — Web Service Container: Apache, Nginx, Reverse Proxy, dan TLS

Nama	Arsyita Devanaya Arianto
NIM	3126640046
Kelas	D4 RPL IT B

Daftar Isi

- 1. Tujuan dan Ruang Lingkup
- 2. Landasan Teori
 - 2.1 Perbandingan Apache dan Nginx
 - 2.2 Peta Konsep: Cara Kerja Apache httpd
 - 2.3 Peta Konsep: Cara Kerja Nginx
 - 2.4 HTTP Routing, Reverse Proxy, Header Forwarding
 - 2.5 TLS: End-to-End versus Offloading, dan Siklus Hidup Sertifikat
 - 2.6 Availability, Health Check, dan Failure Mode
 - 2.7 Logging dan Waktu Standar (UTC)
 - 2.8 Konfigurasi sebagai Kode dan Tata Kelola Perubahan
- 3. Metodologi
- 4. Pelaksanaan Praktikum
 - 4.1 Direktori Kerja
 - 4.2 Sertifikat TLS Laboratorium
 - 4.3 File `compose.yaml`
 - 4.4 File `nginx/conf/default.conf`
 - 4.5 File `apache/sites/index.html`
 - 4.6 File `app/requirements.txt`
 - 4.7 File `app/app.py`
 - 4.8 File `app/Dockerfile`
 - 4.9 Validasi Model Compose
 - 4.10 Build dan Menjalankan Stack
 - 4.11 Pengujian Cepat
 - 4.12 Pemeriksaan Isolasi
 - 4.13 Log
 - 4.14 Checklist PASS
 - 4.15 Cleanup
 - 4.16 Struktur File Akhir
 - 4.17 Ringkasan Hasil Eksekusi
- 5. Verifikasi
- 6. Analisis Hasil
 - 6.1 Masalah yang Muncul dan Cara Mendiagnosisnya
 - 6.2 Risiko Keamanan dan Operasional pada Bab Ini
 - 6.3 Rekomendasi untuk Environment Production-like
- 7. Refleksi Keamanan dan Operasional
- 8. Troubleshooting
- 9. Kesimpulan
- 8. Refleksi Keamanan dan Operasional
- 9. Kesimpulan

1. Tujuan dan Ruang Lingkup

Praktikum Bab 4 bertujuan untuk:

- 1. Menjalankan Apache httpd dan Nginx sebagai web server container dengan konfigurasi custom.
- 2. Membuat virtual host berbasis nama untuk beberapa site lab.
- 3. Menggunakan Nginx sebagai reverse proxy ke backend service.
- 4. Menerapkan sertifikat TLS self-signed untuk simulasi HTTPS.
- 5. Membaca access log dan error log web service dari bind mount.
- 6. Menyusun peta konsep routing, pencegahan header injection, dan pengendalian error pada proxy.

Ruang lingkup bab ini mencakup pembuatan struktur project `~/docker-lab/bab-4` , pembuatan sertifikat self-signed dengan OpenSSL, konfigurasi Nginx sebagai reverse proxy dengan TLS termination dan redirect HTTP ke HTTPS, stack Compose Nginx–Apache–Flask beserta healthcheck, verifikasi alur request client–proxy–backend, serta audit batas Published Port agar hanya proxy yang terekspos ke host.

2. Landasan Teori

2.1 Perbandingan Apache dan Nginx

Bab ini membandingkan dua web server yang paling banyak digunakan secara praktis, yaitu Apache HTTP Server (`httpd`) dan Nginx. Keduanya menjadi baseline image resmi yang kemudian dikustomisasi melalui file konfigurasi, bind mount, atau image turunan.

Aspek	Apache httpd	Nginx
Arsitektur	Process/thread per request	Event-driven asynchronous
Direktori config	<code>/usr/local/apache2/conf</code>	<code>/etc/nginx</code>
Document root	<code>/usr/local/apache2/htdocs</code>	<code>/usr/share/nginx/html</code>
Reverse proxy	<code>mod_proxy</code> (harus di- <i>enable</i>)	<code>proxy_pass</code> (built-in)
Penggunaan memory	Cenderung lebih tinggi (overhead per process/thread)	Lebih hemat, <i>low-level resource efficiency</i>
Use case lab	Virtual host dan kompatibilitas <code>.htaccess</code>	Reverse proxy, static file, TLS termination

Process, thread, dan event-driven. *Process* adalah program yang sedang dieksekusi di memori oleh CPU dan memiliki identitas sendiri (PID). *Thread* adalah unit eksekusi terkecil di dalam suatu proses yang berbagi ruang memori dengan proses induknya. Analogi yang sering dipakai adalah jam analog: aplikasi jam adalah satu *process*, sedangkan jarum detik, jarum menit, dan jarum jam masing-masing bekerja sebagai *thread* terpisah di dalam satu proses tersebut.

Perbedaan penanganan request mengikuti model tersebut. Apache menggunakan *process/thread per request*: setiap request yang masuk ditangani oleh process/thread dalam *pool*, sehingga klien harus menunggu antrian atau ketersediaan thread. Nginx menggunakan model *event-driven asynchronous*: setiap request dianggap sebagai sebuah event baru, dan worker memproses event secara non-blocking. Apabila satu request sedang menunggu (misalnya koneksi ke database), worker yang sama tetap dapat melayani request lain. Konsekuensinya, jumlah thread dan kebutuhan memory pada Nginx tidak naik proporsional terhadap jumlah koneksi aktif.

2.2 Peta Konsep: Cara Kerja Apache httpd

Apache httpd bekerja dengan model *process/thread per request*. Parent process hanya bertugas menerima koneksi, membaca konfigurasi, dan mengelola *pool*; setiap request yang masuk dialokasikan ke satu unit kerja (process atau thread) dan unit tersebut fokus sampai request selesai. Peta konsep berikut memetakan hubungan antara proses, thread, pool, sifat blocking, pilihan MPM, serta peran Apache di dalam lab.



2.3 Peta Konsep: Cara Kerja Nginx



Nginx bekerja dengan model *event-driven asynchronous*. Master process hanya bertugas membaca konfigurasi, membuka socket listener, lalu membuat dan memantau worker. Semua request dikerjakan oleh worker dengan jumlah tetap, dan jumlah worker tidak bergantung pada jumlah koneksi aktif.

Di dalam setiap worker hanya ada satu event loop. Koneksi tidak diperlakukan sebagai process atau thread baru, melainkan dicatat sebagai event di dalam antrean event. Worker memproses antrean itu secara non-blocking: ketika satu event harus menunggu I/O, worker langsung mengembalikan kendali ke event loop untuk mengerjakan koneksi lain yang lebih dulu selesai. Setelah datanya siap, worker melanjutkan event yang tadi tertunda. Dengan begitu setiap koneksi masuk ditangani secara asynchronous sampai seluruh prosesnya selesai.

Event yang datang selalu diterima dan diproses sebanyak mungkin dalam satu putaran. Apabila tidak ada lagi event yang perlu diproses dan koneksi sudah selesai, koneksi ditutup dan worker masuk ke kondisi menunggu, bukan menyisakan satu unit eksekusi khusus untuk tiap koneksi. Inilah alasan Nginx lebih hemat memory dibanding Apache pada tingkat konkurensi yang sama.

2.4 HTTP Routing, Reverse Proxy, Header Forwarding



HTTP memakai pola request–response, dan reverse proxy menjaga konteks request dengan menetapkan header seperti `X-Forwarded-For` , `X-Forwarded-Proto` , `X-Forwarded-Port` , dan `Host` . Header ini hanya dapat dipercaya bila request benar-benar melewati proxy; jika backend dapat diakses langsung, penyerang dapat mengirim *fake header* yang memengaruhi log, URL absolut, dan keputusan keamanan. Karena itu, *trust boundary* dijaga melalui topologi network (backend tanpa *published port*) dan pembatasan proxy terpercaya.

Routing harus dikonfigurasi ketat, karena aturan path yang longgar dapat membuka endpoint administratif, file internal, atau route debugging. Karena itu request lebih dahulu melewati **normalisasi URI** sebelum diteruskan. Kegagalan juga dikendalikan dengan menetapkan kode HTTP yang jelas agar sistem gagal secara terkendali:

Kode	Arti	Pemicu pada lab
301	Moved Permanently	HTTP port 8080 dialihkan ke HTTPS 8443
404	Not Found	Path/URI tidak terdaftar di upstream
405	Method Not Allowed	Method HTTP tidak didukung endpoint
413	Payload Too Large	Body/file melebihi batas ukuran
500	Internal Server Error	Kesalahan pada aplikasi backend
502	Bad Gateway	Upstream tidak dapat dihubungi proxy
503	Service Unavailable	Backend belum siap menerima trafik

2.5 TLS: End-to-End versus Offloading, dan Siklus Hidup Sertifikat

Transport Layer Security melindungi kerahasiaan dan integritas data selama transit serta membantu client memverifikasi identitas server. Ada dua opsi jalur TLS pada arsitektur reverse proxy:

Opsi	Penjelasan	Trade-off
TLS End-to-End	Enkripsi dipasang dari proxy sampai tingkat backend/web app	Backend harus punya trust store; CPU lebih tinggi di tiap hop
TLS Offloading	Enkripsi dihentikan (<i>terminated</i>) hanya di level proxy; komunikasi proxy–backend memakai HTTP biasa	Proxy menjadi titik confianza tunggal; tidak terenkripsi

Praktikum Bab 4 memakai **TLS Offloading** di proxy: Nginx menangani TLS, lalu meneruskan request ke `apache-web:80` dan `flask-app:5000` melalui HTTP di dalam network Compose.

Sertifikat self-signed hanya cocok untuk laboratorium. Pada produksi, sertifikat harus berasal dari *certificate authority* yang dipercaya, memiliki nama host yang tepat, dan diperbarui sebelum kedaluwarsa dengan otomatisasi renewal (misalnya ACME/Let’s Encrypt) di reverse proxy. **Private key bersifat rahasia dan tidak boleh di-hardcode maupun dimasukkan ke dalam Docker Image atau Repository.** Praktik terbaik adalah private key dipasang pada saat runtime sebagai *Secret* atau file *read-only* dengan izin akses/permission paling minimal, dan rotasi perlu diuji tanpa membangun ulang seluruh aplikasi.

Kualitas konfigurasi TLS tidak cukup diukur dari keberhasilan koneksi. Protokol dan cipher usang, redirect yang salah, atau *mixed content* tetap menurunkan perlindungan. Evidence yang memadai mencakup identitas sertifikat, masa berlaku, hasil *handshake*, redirect HTTP-ke-HTTPS, dan kegagalan koneksi ketika hostname tidak cocok.

2.6 Availability, Health Check, dan Failure Mode

Reverse proxy harus membedakan kegagalan koneksi, timeout, dan respons aplikasi yang sah. Timeout yang terlalu panjang menahan worker serta memperbesar risiko *resource exhaustion*, sedangkan timeout yang terlalu pendek menolak request yang sebenarnya valid.

Pemeriksaan kondisi container/layanan dibedakan menjadi dua jenis:

Jenis	Pertanyaan	Aturan Tindakan
Liveness check	Apakah proses di dalam container masih hidup/berjalan?	<i>Pass</i> → container dipertahankan (<i>keep alive</i>). <i>Fail</i> → sistem melakukan restart container.
Readiness check	Apakah layanan siap menerima trafik, termasuk terhubung ke Database, Cache, atau API eksternal?	<i>Pass</i> → trafik diteruskan (<i>pass route</i>). <i>Fail</i> → proxy/upstream memblokir trafik dan mengembalikan <code>503 Service Unavailable</code> .

Healthcheck flask-app pada `compose.yaml` menguji endpoint `/health` menggunakan `urllib.request`, dan proxy memakai `depends_on` dengan `condition: service_healthy` agar trafik tidak diteruskan sebelum backend benar-benar siap. Status container `running` saja tidak membuktikan aplikasi siap menerima request.

2.7 Logging dan Waktu Standar (UTC)

Komponen log penting di proxy meliputi *timestamp* (waktu kejadian), *client IP*, *method*, *path*, *status code*, dan *response time*. Log juga berguna untuk keperluan *forensic*, sehingga field minimum umumnya meliputi waktu, method, path yang sudah disanitasi, status, durasi, ukuran respons, identitas request, dan upstream.

Log **disarankan selalu menggunakan format waktu UTC (Coordinated Universal Time)**. Alasannya, UTC bersifat universal sehingga memudahkan analisis bukti digital antarlokasi server/wilayah — misalnya server di Papua berselisih dua jam dengan WIB — tanpa perlu mengubah pencatatan waktu asli. Pencatatan log tidak boleh merekam secret, token, atau data pribadi secara berlebihan; log merupakan evidence yang harus memiliki *timestamp* konsisten dan retensi yang sesuai.

2.8 Konfigurasi sebagai Kode dan Tata Kelola Perubahan

Konfigurasi reverse proxy diperlakukan sebagai kode: setiap perubahan pada virtual host, route, header, sertifikat, dan timeout melalui *review*, uji sintaks (`nginx -t`), *test* integrasi, dan pencatatan versi. *Rollback* memperhitungkan kompatibilitas aplikasi, cache, sertifikat, dan koneksi aktif. Perubahan pada web layer menghubungkan tim aplikasi, platform, dan keamanan melalui matriks tanggung jawab, dan uji penerimaan dijalankan ulang setelah perubahan dependency atau base image.

3. Metodologi

Laboratorium menggunakan satu host yang menjalankan Docker Engine dan Docker Compose v2 dengan direktori kerja `~/docker-lab/bab-4` yang berisi subdirektori `apache/sites`, `nginx/conf`, `certs`, `logs/nginx`, dan `app`. Sertifikat self-signed RSA 2048 bit dengan SAN `localhost` dan `127.0.0.1` dihasilkan memakai OpenSSL, kemudian permission private key dikunci ke `600`. Konfigurasi Nginx, Compose, dan aplikasi ditulis sebagai file di host dan di-*bind mount* ke container, lalu seluruh service dijalankan melalui Compose pada network `web-net` dengan mode bridge.

Verifikasi dilakukan berurutan mulai dari validasi model Compose (`docker compose config`), pemeriksaan status container, pengujian HTTP/HTTPS dengan `curl`, pemeriksaan `openssl s_client` untuk parameter TLS, audit isolasi network, sampai pembacaan access log pada bind mount. Setiap sub-langkah diuji dengan request positif dan negatif sebelum melanjutkan ke langkah berikutnya. Perintah `cleanup` (`docker compose down`) dijalankan pada akhir sesi agar port 8080 dan 8443 dapat digunakan kembali. Private key tidak dicetak penuh pada laporan.

4. Pelaksanaan Praktikum

4.1 Direktori Kerja

Membuat struktur direktori kerja proyek `~/docker-lab/bab-4` beserta subdirektornya, lalu pindah ke direktori tersebut sebagai lokasi utama praktikum.

```
mkdir -p ~/docker-lab/bab-4/{apache/sites,nginx/conf,certs,logs/nginx,app}
cd ~/docker-lab/bab-4
```

```
ubuntu123@DESKTOP-7A8ST00:~$ mkdir -p ~/docker-lab/bab-4/{apache/sites,nginx/conf,certs,logs/nginx,app}
ubuntu123@DESKTOP-7A8ST00:~$ cd ~/docker-lab/bab-4
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-4$
```

Struktur direktori awal dapat dipastikan dengan `tree`, sebelum file konfigurasi ditulis.

```
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-4$ ls
apache  app  certs  logs  nginx
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-4$ tree
.
├── apache
│   └── sites
├── app
├── certs
├── logs
│   └── nginx
└── nginx
    └── conf
```

Struktur file yang disusun pada bab ini adalah sebagai berikut.

```
bab-4/
├── compose.yaml
├── nginx/conf/default.conf
├── apache/sites/index.html
├── app/Dockerfile
├── app/requirements.txt
├── app/app.py
├── certs/lab.crt
├── certs/lab.key
└── logs/nginx/
```

4.2 Sertifikat TLS Laboratorium

Membuat sertifikat X.509 self-signed RSA 2048 bit berlaku 365 hari untuk `CN=localhost` dengan `subjectAltName` agar dapat diakses melalui `localhost` maupun `127.0.0.1`. Private key langsung dikunci dengan permission `600` agar tidak dapat dibaca service lain, sedangkan sertifikat publik diberi permission `644`.

```
openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
-keyout certs/lab.key \
-out certs/lab.crt \
-subj "/CN=localhost/O=DevSecOps Docker Lab" \
-addext "subjectAltName=DNS:localhost,IP:127.0.0.1"
```

```
chmod 600 certs/lab.key
chmod 644 certs/lab.crt
```

```
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-4$ openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
-keyout certs/lab.key \
-out certs/lab.crt \
-subj "/CN=localhost/O=DevSecOps Docker Lab" \
-addext "subjectAltName=DNS:localhost,IP:127.0.0.1"
-----
+-----+
| Subject: CN=localhost, O=DevSecOps Docker Lab |
+-----+
| Subject Alternative Name: DNS:localhost, IP:127.0.0.1 |
+-----+
-----
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-4$ chmod 600 certs/lab.key
chmod: cannot access 'certs/lab.key': No such file or directory
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-4$ chmod 600 certs/lab.key
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-4$ chmod 644 certs/lab.crt
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-4$
```

Pada eksekusi pertama, `chmod 600 certs/lab.key` gagal dengan pesan `cannot access 'certs/lab.key': No such file or directory`. Penyebabnya adalah perintah yang dijalankan salah `certs/lab.ke` tidak ditemukan. Setelah `certs/lab.key`, kedua perintah `chmod` berhasil dijalankan.

Catatan: `lab.key` merupakan private key. File ini tidak boleh dikomit ke repository dan hanya dipasang saat runtime sebagai mount read-only.

4.3 File `compose.yaml`

Menulis `compose.yaml` sebagai definisi deklaratif stack berisi 3 service (`proxy` = Nginx, `apache-web` = Apache httpd, `flask-app` = Flask) dan 1 network bridge `web-net` . Hanya `proxy` yang memiliki published port, dan kedua port dibatasi ke loopback `127.0.0.1` sehingga tidak mengikat seluruh interface host.

```
nano compose.yaml
```

```
services:
  proxy:
    image: nginx:alpine
    ports:
      - "127.0.0.1:8080:80"
      - "127.0.0.1:8443:443"
    volumes:
      - ./nginx/conf:/etc/nginx/conf.d:ro
      - ./certs:/etc/nginx/certs:ro
      - ./logs/nginx:/var/log/nginx
    networks:
      - web-net
    depends_on:
      apache-web:
        condition: service_started
      flask-app:
        condition: service_healthy

  apache-web:
    image: httpd:2.4-alpine
    volumes:
      - ./apache/sites:/usr/local/apache2/htdocs:ro
    networks:
      - web-net

  flask-app:
    build:
      context: ./app
    networks:
      - web-net
    healthcheck:
      test:
        - CMD
        - python
        - -c
        - "import urllib.request; urllib.request.urlopen('http://127.0.0.1:5000/health')"
      interval: 5s
      timeout: 3s
      retries: 5
      start_period: 5s

networks:
  web-net:
    driver: bridge
```

```

ubuntu123@DESKTOP-7ABST00:~/docker-lab/bab-4$ nano compose.yml
ubuntu123@DESKTOP-7ABST00:~/docker-lab/bab-4$ cat compose.yml
services:
  proxy:
    image: nginx:alpine
    ports:
      - "127.0.0.1:8080:80"
      - "127.0.0.1:8443:443"
    volumes:
      - ./nginx/conf:/etc/nginx/conf.d:ro
      - ./certs:/etc/nginx/certs:ro
      - ./logs/nginx:/var/log/nginx
    networks:
      - web-net
    depends_on:
      apache-web:
        condition: service_started
      flask-app:
        condition: service_healthy

  apache-web:
    image: httpd:2.4-alpine
    volumes:
      - ./apache/sites:/usr/local/apache2/htdocs:ro
    networks:
      - web-net

  flask-app:
    build:
      context: ./app
    networks:
      - web-net
    healthcheck:
      test:
        - CMD
        - python
        - -c
        - "import urllib.request; urllib.request.urlopen('http://127.0.0.1:5000/health')"
      interval: 5s
      timeout: 3s
      retries: 5
      start_period: 5s

networks:
  web-net:
    driver: bridge

```

`./certs` dan `./nginx/conf` dipasang read-only (`:ro`) karena container hanya membutuhkan akses baca; `./logs/nginx` dipasang *writable* karena Nginx harus menulis access log dan error log ke direktori host. Image `nginx:alpine` sudah membawa modul `stream` sehingga `proxy_pass` tersedia tanpa konfigurasi tambahan.

4.4 File `nginx/conf/default.conf`

Nginx dikonfigurasi sebagai reverse proxy untuk dua upstream, yaitu `apache_backend` (`apache-web:80`) dan `flask_backend` (`flask-app:5000`). Port 80 hanya berfungsi sebagai pintu masuk yang mengembalikan `301` ke HTTPS, sedangkan port 443 menjalankan TLS termination dengan protokol minimum TLS 1.2.

```
nano nginx/conf/default.conf
```

```

upstream apache_backend {
    server apache-web:80;
}

upstream flask_backend {
    server flask-app:5000;
}

server {
    listen 80;
    server_name localhost;
    return 301 https://$host:8443$request_uri;
}

server {
    listen 443 ssl;
    server_name localhost;

    ssl_certificate      /etc/nginx/certs/lab.crt;
    ssl_certificate_key  /etc/nginx/certs/lab.key;
    ssl_protocols TLSv1.2 TLSv1.3;

    add_header X-Content-Type-Options "nosniff" always;
    add_header X-Frame-Options "DENY" always;
    add_header Referrer-Policy "no-referrer" always;

    location / {
        proxy_pass http://apache_backend;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    location /api/ {
        proxy_pass http://flask_backend;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}

```

```

ubuntu123@DESKTOP-7AB5T00:~/docker-lab/bab-4$ nano nginx/conf/default.conf
ubuntu123@DESKTOP-7AB5T00:~/docker-lab/bab-4$ cat nginx/conf/default.conf
upstream apache_backend {
    server apache-web:80;
}

upstream flask_backend {
    server flask-app:5000;
}

server {
    listen 80;
    server_name localhost;
    return 301 https://$host:8443$request_uri;
}

server {
    listen 443 ssl;
    server_name localhost;

    ssl_certificate      /etc/nginx/certs/lab.crt;
    ssl_certificate_key  /etc/nginx/certs/lab.key;
    ssl_protocols TLSv1.2 TLSv1.3;

    add_header X-Content-Type-Options "nosniff" always;
    add_header X-Frame-Options "DENY" always;
    add_header Referrer-Policy "no-referrer" always;

    location / {
        proxy_pass http://apache_backend;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    location /api/ {
        proxy_pass http://flask_backend/;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
ubuntu123@DESKTOP-7AB5T00:~/docker-lab/bab-4$ |

```

Beberapa keputusan konfigurasi yang perlu dicatat:

1. **Virtual host berbasis nama.** Blok `server` dipisahkan berdasarkan `server_name`, sehingga penambahan site lab berikutnya cukup dengan blok `server` baru tanpa mengubah konfigurasi service lain.
2. **Header terpercaya diteruskan.** `Host`, `X-Real-IP`, `X-Forwarded-For`, dan `X-Forwarded-Proto` diisi proxy agar backend mengetahui identitas client asli dan skema koneksi yang sebenarnya. Karena `apache-web` dan `flask-app` tidak memiliki published port, satu-satunya jalan masuk ke kedua service tersebut adalah melalui proxy.
3. **Normalisasi trailing slash.** `location /api/` dipasangkan dengan `proxy_pass http://flask_backend/` (memakai trailing slash) sehingga prefix `/api` dihapus sebelum request diteruskan ke Flask.
4. **Header keamanan respons.** `X-Content-Type-Options`, `X-Frame-Options`, dan `Referrer-Policy` dipasang dengan flag `always` agar tetap terkirim pada respons error seperti 4xx/5xx.

4.5 File `apache/sites/index.html`

Apache httpd berfungsi sebagai origin server yang menyajikan halaman statis dari document root `/usr/local/apache2/htdocs`, yang di-*bind mount* read-only dari `./apache/sites`. Halaman ini menjadi bukti bahwa request benar-benar melewati proxy menuju Apache, bukan dilayani langsung oleh Nginx.

```
nano apache/sites/index.html
```

```

<!DOCTYPE html>
<html lang="id">
<head>
    <meta charset="UTF-8">
    <title>DevSecOps Bab 4</title>
</head>
<body>
    <h1>Apache di Belakang Nginx</h1>
    <p>Halaman Bab 4 berhasil dilayani melalui reverse proxy.</p>
    <p><a href="/api/">Uji Flask API</a></p>
</body>
</html>

```

```

ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-4$ nano apache/sites/index.html
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-4$ cat apache/sites/index.html
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <title>DevSecOps Bab 4</title>
</head>
<body>
  <h1>Apache di Belakang Nginx</h1>
  <p>Halaman Bab 4 berhasil dilayani melalui reverse proxy.</p>
  <p><a href="/api/">Uji Flask API</a></p>
</body>
</html>

```

4.6 File app/requirements.txt

Mendaftarkan dependensi Python untuk aplikasi Flask beserta WSGI server `gunicorn`. Pin versi dipakai agar hasil build reproducible dan tidak berubah sesuai rilis terbaru.

```
nano app/requirements.txt
```

```

Flask==3.1.2
gunicorn==23.0.0

```

```

ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-4$ nano app/requirements.txt
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-4$ cat app/requirements.txt
Flask==3.1.2
gunicorn==23.0.0
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-4$

```

4.7 File app/app.py

Aplikasi Flask menyediakan dua endpoint JSON. Endpoint `/` mengembalikan informasi service termasuk nilai `X-Forwarded-Proto` yang diteruskan proxy — nilai ini membuktikan bahwa header tepercaya benar-benar sampai di backend — sedangkan `/health` menjadi target *readiness check* Compose.

```
nano app/app.py
```

```
from flask import Flask, jsonify, request
```

```
app = Flask(__name__)
```

```

@app.get("/")
def index():
    return jsonify(
        status="ok",
        service="flask-app",
        message="API Bab 4 berhasil diakses melalui Nginx",
        forwarded_proto=request.headers.get("X-Forwarded-Proto"),
    )

```

```

@app.get("/health")
def health():
    return jsonify(status="healthy"), 200

```

```

ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-4$ nano app/app.py
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-4$ cat app/app.py
from flask import Flask, jsonify, request

app = Flask(__name__)

@app.get("/")
def index():
    return jsonify(
        status="ok",
        service="flask-app",
        message="API Bab 4 berhasil diakses melalui Nginx",
        forwarded_proto=request.headers.get("X-Forwarded-Proto"),
    )

@app.get("/health")
def health():
    return jsonify(status="healthy"), 200
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-4$ |

```

4.8 File app/Dockerfile

Image aplikasi dibangun dari `python:3.12-slim`. Container berjalan sebagai user non-root `appuser` dengan UID 10001, bukan sebagai root, sehingga probabilitas dampak celah eksekusi kode jauh lebih kecil. Build dik resiliency terhadap kegagalan jaringan dengan `pip install --no-cache-dir`.

```
nano app/Dockerfile
```

```
FROM python:3.12-slim
```

```
WORKDIR /app
```

```
COPY requirements.txt .
```

```
RUN pip install --no-cache-dir -r requirements.txt
```

```
COPY app.py .
```

```
RUN useradd --system --uid 10001 --no-create-home appuser
```

```
USER appuser
```

```
EXPOSE 5000
```

```
CMD ["gunicorn", "--bind", "0.0.0.0:5000", "--workers", "2", "app:app"]
```

```

ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-4$ nano app/Dockerfile
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-4$ cat app/Dockerfile
FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY app.py .

RUN useradd --system --uid 10001 --no-create-home appuser
USER appuser

EXPOSE 5000

CMD ["gunicorn", "--bind", "0.0.0.0:5000", "--workers", "2", "app:app"]
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-4$

```

Catatan: `EXPOSE 5000` hanya bersifat dokumentatif dan tidak memublikasikan port ke host. Pem publikasian tetap harus dinyatakan eksplisit pada `ports`.

4.9 Validasi Model Compose

Sebelum menjalankan apa pun, seluruh file yang ditulis diverifikasi. Langkah pertama `find` untuk memastikan tidak ada file yang terlewat atau salah nama.

```
cd ~/docker-lab/bab-4
find . -maxdepth 3 -type f | sort
```

```
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-4$ find . -maxdepth 3 -type f | sort
./apache/sites/index.html
./app/Dockerfile
./app/app.py
./app/requirements.txt
./certs/lab.crt
./certs/lab.key
./compose.yml
./nginx/conf/default.conf
```

Delapan file terverifikasi ada: compose.yml, apache/sites/index.html, app/Dockerfile, app/app.py, app/requirements.txt, certs/lab.crt, certs/lab.key, dan nginx/conf/default.conf.

docker compose config memvalidasi kebenaran YAML sekaligus menampilkan model komposisi akhir setelah *interpolation* dan *merge*. Keluarannya juga membuktikan bahwa hanya proxy yang memiliki ports, sedangkan apache-web dan flask-app tidak memiliki published port sama sekali.

docker compose config

```
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-4$ docker compose config
name: bab-4
services:
  apache-web:
    image: httpd:2.4-alpine
    networks:
      web-net: null
    volumes:
      - type: bind
        source: /home/ubuntu123/docker-lab/bab-4/apache/sites
        target: /usr/local/apache2/htdocs
        read_only: true
        bind: {}
  flask-app:
    build:
      context: /home/ubuntu123/docker-lab/bab-4/app
      dockerfile: Dockerfile
    healthcheck:
      test:
        - CMD
        - python
        - -c
        - import urllib.request; urllib.request.urlopen('http://127.0.0.1:5000/health')
      timeout: 3s
      interval: 5s
      retries: 5
      start_period: 5s
    networks:
      web-net: null
  proxy:
    depends_on:
      apache-web:
        condition: service_started
        required: true
      flask-app:
        condition: service_healthy
        required: true
    image: nginx:alpine
    networks:
      web-net: null
    ports:
      - mode: ingress
        host_ip: 127.0.0.1
        target: 80
        published: "8080"
        protocol: tcp
      - mode: ingress
        host_ip: 127.0.0.1
        target: 443
        published: "8443"
        protocol: tcp
    volumes:
      - type: bind
        source: /home/ubuntu123/docker-lab/bab-4/nginx/conf
        target: /etc/nginx/conf.d
        read_only: true
        bind: {}
      - type: bind
        source: /home/ubuntu123/docker-lab/bab-4/certs
        target: /etc/nginx/certs
        read_only: true
        bind: {}
      - type: bind
        source: /home/ubuntu123/docker-lab/bab-4/logs/nginx
        target: /var/log/nginx
        bind: {}
    networks:
      web-net:
        name: bab-4_web-net
        driver: bridge
```

--services hanya mendaftarkan nama-nama service yang terdefinisi tanpa menampilkan konfigurasi lengkap.

docker compose config --services

```
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-4$ docker compose config --services
apache-web
flask-app
proxy
```

Service yang harus muncul:

Urutan keluaran `docker compose config --services` bersifat alfabetis (`apache-web` , `flask-app` , `proxy`) dan tetap terdiri dari tiga service yang sama dengan cheatsheet.

Stack dijalankan dengan membangun image aplikasi sekaligus menyalakan semua container di background sesuai model Compose.

[illegible]

```
docker compose ps
```

flask-app berstatus healthy setelah healthcheck berhasil, sementara proxy dan apache-web berstatus running. Notice flask-app sudah *Started* sebelum proxy selesai *Starting*. Hal ini menunjukkan depends_on dengan condition: service_healthy bekerja: proxy menunggu backend siap, bukan sekadar aktif.

[illegible]

Pengujian dilakukan dengan request positif dan negatif untuk memastikan alur client-proxy-backend berjalan sesuai mestinya.

```
curl -I http://localhost:8080/
```

```
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-4$ curl -I http://localhost:8080/
HTTP/1.1 301 Moved Permanently
Server: nginx/1.31.5
Date: Fri, 02 Oct 2026 05:26:06 GMT
Content-Type: text/html
Content-Length: 169
Connection: keep-alive
Location: https://localhost:8443/
```

Respons 301 Moved Permanently dengan header Location: https://localhost:8443/ membuktikan konfigurasi redirect bekerja dan tidak ada mixed content.

```
curl -I http://localhost:8080/
```

```
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-4$ curl -I http://localhost:8080/
HTTP/1.1 301 Moved Permanently
Server: nginx/1.31.5
Date: Fri, 02 Oct 2026 05:26:06 GMT
Content-Type: text/html
Content-Length: 169
Connection: keep-alive
Location: https://localhost:8443/
```

Uji 2 — Halaman Apache melalui Nginx (port 8443). Flag `-k` dipakai karena sertifikat bersifat self-signed.

```
curl -k -i https://localhost:8443/
```

```
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-4$ curl -k -i https://localhost:8443/
HTTP/1.1 200 OK
Server: nginx/1.31.5
Date: Fri, 02 Oct 2026 05:29:05 GMT
Content-Type: text/html
Content-Length: 282
Connection: keep-alive
Last-Modified: Fri, 02 Oct 2026 05:16:57 GMT
ETag: "31a-65cd4a4275eac"
Accept-Ranges: bytes
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
Referrer-Policy: no-referrer

<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <title>DevSecOps Bab 4</title>
</head>
<body>
  <h1>Apache di Belakang Nginx</h1>
  <p>Halaman Bab 4 berhasil dilayani melalui reverse proxy.</p>
  <p><a href="/api/">Uji Flask API</a></p>
</body>
</html>
```

Respons 200 OK dengan body HTML “Apache di Belakang Nginx” membuktikan request berhasil melewati proxy menuju origin server Apache. Header X-Frame-Options: DENY, X-Content-Type-Options: nosniff, dan Referrer-Policy: no-referrer juga terlihat pada respons.

Uji 3 — Flask API melalui Nginx.

```
curl -k https://localhost:8443/api/
```

```
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-4$ curl -k https://localhost:8443/api/
{"forwarded_proto":"https","message":"API Bab 4 berhasil diakses melalui Nginx","service":"flask-app","status":"ok"}
```

Field `forwarded_proto` bernilai `https`. Ini bukti langsung bahwa `proxy_set_header X-Forwarded-Proto $scheme` bekerja dan aplikasi backend membaca nilai yang benar — bukan tebakan dari `request.scheme`.

Uji 4 — Health endpoint Flask. Endpoint ini juga dipakai healthcheck Compose.

```
curl -k -i https://localhost:8443/api/health
```

```
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-4$ curl -k -i https://localhost:8443/api/health
HTTP/1.1 200 OK
Server: nginx/1.31.5
Date: Fri, 02 Oct 2026 05:29:20 GMT
Content-Type: application/json
Content-Length: 21
Connection: keep-alive
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
Referrer-Policy: no-referrer
{"status":"healthy"}
```

Uji 5 — Informasi parameter TLS. `openssl s_client` dipakai untuk memastikan protokol dan cipher yang benar-benar dinegosiasikan, bukan sekadar mengasumsikan koneksi berhasil.

```
openssl s_client \
  -connect localhost:8443 \
  -servername localhost \
  -brief </dev/null
```

```
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-4$ openssl s_client \
  -connect localhost:8443 \
  -servername localhost \
  -brief </dev/null
Connecting to 127.0.0.1
depth=0 CN=localhost, O=DevSecOps Docker Lab
verify error:num=18:self-signed certificate
CONNECTION ESTABLISHED
Protocol version: TLSv1.3
Ciphersuite: TLS_AES_256_GCM_SHA384
Peer certificate: CN=localhost, O=DevSecOps Docker Lab
Hash used: SHA256
Signature type: rsa_pss_rsae_sha256
Verification error: self-signed certificate
Negotiated TLS1.3 group: X25519MLKEM768
DONE
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-4$
```

Hasil *negotiation* menunjukkan Protocol version: TLSv1.3 , Ciphersuite: TLS_AES_256_GCM_SHA384 , dan verification error self-signed certificate yang memang diharapkan di laboratorium. Field Verification: verify certificate memastikan nama host localhost sesuai dengan CN dan subjectAltName pada sertifikat.

Catatan: curl -k hanya digunakan untuk mempercayai sertifikat self-signed laboratorium. Pada lingkungan production-like, sertifikat dari CA terpercaya tidak memerlukan opsi ini.

4.12 Pemeriksaan Isolasi

Bagian ini membuktikan bahwa hanya proxy yang terekspos ke host dan kedua backend hanya dapat dijangkau dari dalam web-net .

```
docker compose ps
```

```
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-4$ docker compose ps
NAME                STATUS                                PORTS
bab-4-proxy-web-1   http://127.0.0.1:8443               8443/tcp
bab-4-flask-app-1   bab-4-flask-app-1                   5000/tcp
bab-4-apache-web-1  bab-4-apache-web-1                  80/tcp
```

Kolom PORTS hanya terisi pada proxy ; apache-web dan flask-app tidak memiliki published port. Keduanya tetap dapat menjalankan service dengan normal karena komunikasi berlangsung melalui network internal Compose.

Struktur network beserta container yang terhubung dan IP masing-masing dapat diperiksa untuk memastikan topologi segmentasi.

```
docker network inspect bab-4_web-net
```

```

ubuntu123@DESKTOP-7A85T00:~/docker-lab/bab-4$ docker network inspect bab-4_web-net
{
  "Name": "bab-4_web-net",
  "Id": "627f9a54dc1fce1f818be5315d96c21b217202eac5534adcf40c3dd1c13ddcb",
  "Created": "2020-10-02T12:24:33.163048996+07:00",
  "Scope": "local",
  "Driver": "bridge",
  "EnableIPv6": true,
  "EnableIPAM": false,
  "IPAM": {
    "Driver": "default",
    "Options": null,
    "Config": [
      {
        "Subnet": "172.21.0.0/16",
        "Gateway": "172.21.0.1"
      }
    ]
  },
  "Internal": false,
  "Attachable": false,
  "Ingress": false,
  "ConfigFrom": {
    "Network": ""
  },
  "ConfigOnly": false,
  "Options": {},
  "Labels": {
    "com.docker.compose.config-hash": "9c7bc8f92788b81a904f62f24858cde1fac96f9c7e824885e77a0dd33a025a98",
    "com.docker.compose.network": "web-net",
    "com.docker.compose.project": "bab-4",
    "com.docker.compose.version": "5.5.1"
  },
  "Containers": {
    "3cc7acc9b48406f5607ac883cf84062f68011b47cc3b3b566d24626d51b8a8": {
      "Name": "bab-4-flask-app-1",
      "EndpointID": "a9db93150fb910c1161d66774b18dc10fcc13b8cbafba856947cc0fe6892538",
      "Networks": {
        "web-net": {
          "IPAddress": "172.21.0.3/16",
          "IPv6Address": ""
        }
      }
    },
    "839c6d77b704f62ae323d18db7732736896f6c2ec547abc64bb17f3938c95b89": {
      "Name": "bab-4-apache-web-1",
      "EndpointID": "cda37431b1c86976bcedb8348f7baab74fce2c6bc927a827c3bc8ab8e2ecda",
      "Networks": {
        "web-net": {
          "IPAddress": "172.21.0.2/16",
          "IPv6Address": ""
        }
      }
    },
    "cb823fc2f08b118bc8d5f408f23f3585dec06883837ae5237c6b7e9b7dced8": {
      "Name": "bab-4-proxy-1",
      "EndpointID": "5401c84488803e8156890e813a1235670a1048b5faa1377ac5fbd0deb266c5",
      "Networks": {
        "web-net": {
          "IPAddress": "172.21.0.4/16",
          "IPv6Address": ""
        }
      }
    }
  },
  "Status": {
    "IPAM": {
      "Subnet": {
        "172.21.0.0/16": {
          "IPsInUse": 4,
          "DynamicIPsAvailable": 65536
        }
      }
    }
  }
}

```

Network bab-4_web-net bertipe bridge dengan tiga container anggota: bab-4-apache-web-1 , bab-4-proxy-1 , dan bab-4-flask-app-1 . Ketiganya berbagi satu segment jaringan internal.

Dari dalam container proxy, origin server Apache dapat dicapai langsung melalui DNS service tanpa melewati host.

```
docker compose exec proxy wget -qO- http://apache-web/
```

```

ubuntu123@DESKTOP-7A85T00:~/docker-lab/bab-4$ docker compose exec proxy wget -qO- http://apache-web/
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <title>DevSecOps Bab 4</title>
</head>
<body>
  <h1>Apache di Belakang Nginx</h1>
  <p>Halaman Bab 4 berhasil dilayani melalui reverse proxy.</p>
  <p><a href="/api/">Uji Flask API</a></p>
</body>
</html>

```

Backend Flask dan endpoint health-nya juga dapat diuji dari dalam container proxy.

```
docker compose exec proxy wget -qO- http://flask-app:5000/health
```

```

ubuntu123@DESKTOP-7A85T00:~/docker-lab/bab-4$ docker compose exec proxy wget -qO- http://flask-app:5000/health
{"status":"healthy"}
ubuntu123@DESKTOP-7A85T00:~/docker-lab/bab-4$

```

Dua pengujian tersebut membuktikan resolusi nama service bekerja pada user-defined bridge network sekaligus membuktikan isolasi jaringan: container di luar web-net tidak akan dapat mencapai kedua backend.

4.13 Log

Log proxy dibaca per service untuk memastikan tidak ada error upstream selama pengujian.

```

docker compose logs proxy
docker compose logs apache-web
docker compose logs flask-app

```

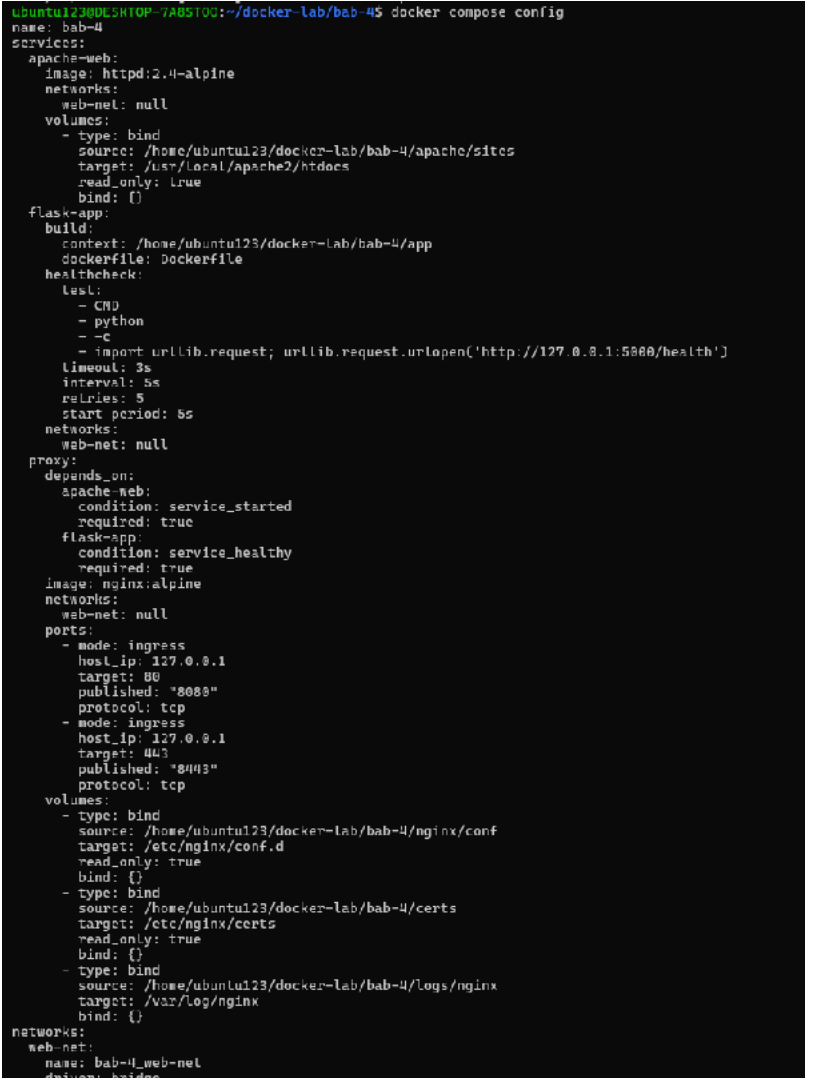
[illegible]

Access log dan error log dapat dibaca langsung dari bind mount di host karena `./logs/nginx` dipasang *writable* oleh Nginx, sehingga datanya tetap tersedia setelah container dihentikan.

```
tail -n 20 logs/nginx/access.log
tail -n 20 logs/nginx/error.log
```

Setiap baris access log memuat `timestamp` dalam UTC, `client IP`, `method`, `path`, `status code`, dan `user agent`. Error log tidak menunjukkan error upstream maupun kegagalan handshake.

4.14 Checklist PASS

No	Checklist PASS	Evidence
1	docker compose config tidak menghasilkan galat	 <pre> ubuntu123@DLSKTOP-7A85T0Q:~/docker-lab/bab-4\$ docker compose config name: bab-4 services: apache-web: image: httpd:2.4-alpine networks: web-net: null volumes: - type: bind source: /home/ubuntu123/docker-lab/bab-4/apache/sites target: /usr/local/apache2/htdocs read_only: true bind: {} flask-app: build: context: /home/ubuntu123/docker-lab/bab-4/app dockerfile: Dockerfile healthcheck: test: - CMD - python - -c - import urllib.request; urllib.request.urlopen('http://127.0.0.1:5000/health') timeout: 3s interval: 5s retries: 5 start_period: 5s networks: web-net: null proxy: depends_on: apache-web: condition: service_started required: true flask-app: condition: service_healthy required: true image: nginx:alpine networks: web-net: null ports: - mode: ingress host_ip: 127.0.0.1 target: 80 published: "8083" protocol: tcp - mode: ingress host_ip: 127.0.0.1 target: 443 published: "8443" protocol: tcp volumes: - type: bind source: /home/ubuntu123/docker-lab/bab-4/nginx/conf target: /etc/nginx/conf.d read_only: true bind: {} - type: bind source: /home/ubuntu123/docker-lab/bab-4/certs target: /etc/nginx/certs read_only: true bind: {} - type: bind source: /home/ubuntu123/docker-lab/bab-4/logs/nginx target: /var/log/nginx bind: {} networks: web-net: name: bab-4_web-net driver: bridge </pre>

No	Checklist PASS	Evidence
2	flask-app berstatus healthy	<pre>ubuntu123@DESKTOP-7A85T00:~/docker-lab/bab-4\$ docker compose ps NAME STATUS CMPTYPE SERVICE CREATED STATUS PORTS bab-4-apache-web-1 Up 2h nginx apache-web 11 minutes ago Up 10 minutes 80/tcp bab-4-flask-app-1 Up 2h flask flask-app 11 minutes ago Up 10 minutes 5000/tcp bab-4-proxy-1 Up 2h proxy proxy 11 minutes ago Up 10 minutes 127.0.0.1:8080->8080/tcp, 127.0.0.1:8443->443/tcp</pre>
3	HTTP port 8080 menghasilkan redirect 301	<pre>ubuntu123@DESKTOP-7A85T00:~/docker-lab/bab-4\$ curl -I http://localhost:8080/ HTTP/1.1 301 Moved Permanently Server: nginx/1.31.5 Date: Fri, 02 Oct 2026 05:39:37 GMT Content-Type: text/html Content-Length: 169 Connection: keep-alive Location: https://localhost:8443/</pre>
4	HTTPS port 8443 menampilkan halaman Apache	<pre>ubuntu123@DESKTOP-7A85T00:~/docker-lab/bab-4\$ curl -k -i https://localhost:8443/ HTTP/1.1 200 OK Server: nginx/1.31.5 Date: Fri, 02 Oct 2026 05:40:30 GMT Content-Type: text/html Content-Length: 292 Connection: keep-alive Last-Modified: Fri, 02 Oct 2026 05:16:57 GMT ETag: "11a-65cd4b275e4e" Accept-Ranges: bytes X-Content-Type-Options: nosniff X-Frame-Options: DENY Referer-Policy: no-referrer <!DOCTYPE html> <html lang="id"> <head> <meta charset="UTF-8"> <title>DevSecOps Bab 4</title> </head> <body> <h1>Apache di Belakang Nginx</h1> <p>Halaman Bab 4 berhasil dilayani melalui reverse proxy.</p> <p>Uji Flask API</p> </body> </html></pre>
5	/api/ menghasilkan JSON Flask	<pre>ubuntu123@DESKTOP-7A85T00:~/docker-lab/bab-4\$ curl -k https://localhost:8443/api/ {"forwarded_proto":"https","message":"API Bab 4 berhasil diakses melalui Nginx","service":"Flask-app","status":"on"}</pre>
6	TLS 1.2 atau TLS 1.3 berhasil dinegosiasikan	<pre>ubuntu123@DESKTOP-7A85T00:~/docker-lab/bab-4\$ openssl s_client -connect localhost:8443 -servername localhost -brief -dev/null Connecting to 127.0.0.1: depth=0 CN=localhost, O=DevSecOps Docker Lab verify error:num=10:self-signed certificate CONNECTION ESTABLISHED Protocol version: TLSv1.3 Cipher suite: TLS_AES_256_GCM_SHA384 Peer certificate: CN=localhost, O=DevSecOps Docker Lab Hash used: SHA256 Signature type: RSA_PSS_256_SHA256 Verification error: self-signed certificate Negotiated TLS 1.3 group: X25519MLKEM768 DONE</pre>
7	Apache dan Flask tidak memiliki published port	<pre>ubuntu123@DESKTOP-7A85T00:~/docker-lab/bab-4\$ docker compose ps NAME STATUS CMPTYPE SERVICE CREATED STATUS PORTS bab-4-apache-web-1 Up 2h nginx apache-web 20 minutes ago Up 10 minutes 80/tcp bab-4-flask-app-1 Up 2h flask flask-app 18 minutes ago Up 10 minutes 5000/tcp bab-4-proxy-1 Up 2h proxy proxy 18 minutes ago Up 10 minutes 127.0.0.1:8080->8080/tcp, 127.0.0.1:8443->443/tcp</pre>
8	Request tercatat pada access log Nginx	<pre>202.10.1.1 - - [02/Oct/2026:05:40:30 +0000] "GET / HTTP/1.1" 301 169 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/138.0.0.0 Safari/537.36" 202.10.1.1 - - [02/Oct/2026:05:40:30 +0000] "GET / HTTP/1.1" 200 292 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/138.0.0.0 Safari/537.36" 202.10.1.1 - - [02/Oct/2026:05:41:30 +0000] "GET /api/ HTTP/1.1" 200 21 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/138.0.0.0 Safari/537.36" 202.10.1.1 - - [02/Oct/2026:05:41:30 +0000] "GET / HTTP/1.1" 301 169 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/138.0.0.0 Safari/537.36" 202.10.1.1 - - [02/Oct/2026:05:41:30 +0000] "GET / HTTP/1.1" 200 292 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/138.0.0.0 Safari/537.36"</pre>

Seluruh butir checklist terpenuhi dengan status PASS .

4.15 Cleanup

Perintah cleanup menghentikan dan menghapus container serta network proyek. Port 8080 dan 8443 kembali bebas untuk dipakai bab berikutnya.

```
cd ~/docker-lab/bab-4
docker compose down

ubuntu123@DESKTOP-7A85T00:~/docker-lab/bab-4$ docker compose down
[+] down 4/4
✓ Container bab-4-proxy-1      Removed
✓ Container bab-4-apache-web-1 Removed
✓ Container bab-4-flask-app-1  Removed
✓ Network bab-4_web-net        Removed
ubuntu123@DESKTOP-7A85T00:~/docker-lab/bab-4$ docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED       STATUS       PORTS   NAMES
ubuntu123@DESKTOP-7A85T00:~/docker-lab/bab-4$
```

Tiga container (apache-web , flask-app , proxy) dan network bab-4_web-net terhapus. Perintah docker ps setelahnya tidak menampilkan container yang berjalan.

Image lokal hasil build dapat dihapus bila diperlukan:

```
docker compose down --rmi local
```

4.16 Struktur File Akhir

Susunan file proyek setelah seluruh langkah selesai sebagai dokumentasi hasil akhir.

```
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-4$ tree
.
├── apache
│   └── sites
│       └── index.html
├── app
│   ├── Dockerfile
│   ├── app.py
│   └── requirements.txt
├── certs
│   ├── lab.crt
│   └── lab.key
├── compose.yml
├── logs
│   └── nginx
│       ├── access.log
│       └── error.log
└── nginx
    └── conf
        └── default.conf

9 directories, 10 files
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-4$
```

Terdapat 9 directory dan 10 file, yaitu `apache/sites/index.html` , `app/Dockerfile` , `app/app.py` , `app/requirements.txt` , `certs/lab.crt` , `certs/lab.key` , `compose.yml` , `logs/nginx/access.log` , `logs/nginx/error.log` , dan `nginx/conf/default.conf` . Perhatikan bahwa `access.log` dan `error.log` baru muncul pada struktur akhir karena baru terbentuk setelah Nginx menerima request.

4.17 Ringkasan Hasil Eksekusi

Perintah	Hasil / Catatan	Status
<code>docker compose config</code>	Model Compose valid; hanya <code>proxy</code> yang memiliki published port	PASS
<code>docker compose config --services</code>	Menghasilkan <code>apache-web</code> , <code>flask-app</code> , <code>proxy</code>	PASS
<code>docker compose up -d --build</code>	Build image berhasil, 3 container berjalan pada <code>web-net</code>	PASS
<code>docker compose ps</code>	<code>flask-app</code> <code>healthy</code> , <code>proxy</code> dan <code>apache-web</code> <code>running</code>	PASS
<code>curl -I http://localhost:8080/</code>	301 Moved Permanently → <code>https://localhost:8443/</code>	PASS
<code>curl -k -i https://localhost:8443/</code>	200 OK , halaman “Apache di Belakang Nginx” terlayani	PASS
<code>curl -k https://localhost:8443/api/</code>	JSON Flask dengan <code>forwarded_proto: https</code>	PASS
<code>curl -k -i https://localhost:8443/api/health</code>	200 OK dengan <code>{"status": "healthy"}</code>	PASS
<code>openssl s_client -brief</code>	Protocol version: TLSv1.3 , TLS_AES_256_GCM_SHA384	PASS
<code>docker network inspect bab-4_web-net</code>	3 container anggota pada segment bridge internal	PASS
<code>docker compose exec proxy wget -qO- http://apache-web/</code>	HTML Apache terbaca dari dalam container <code>proxy</code>	PASS
<code>docker compose exec proxy wget -qO- http://flask-app:5000/health</code>	<code>{"status": "healthy"}</code> dari backend internal	PASS
<code>tail -n 20 logs/nginx/access.log</code>	Request tercatat pada bind mount host	PASS
<code>docker compose down</code>	3 container dan 1 network terhapus	PASS

5. Verifikasi

Skenario Pengujian	Hasil
Nginx reverse proxy dapat mengakses Apache melalui nama service	PASS
Endpoint <code>/api</code> diarahkan ke backend Flask	PASS
HTTPS self-signed berjalan di port 8443	PASS
HTTP port 8080 dialihkan otomatis ke HTTPS	PASS
Parameter TLS yang dinegosiasikan adalah TLS 1.3 dengan cipher modern	PASS

Skenario Pengujian	Hasil
X-Forwarded-Proto terkirim dan dibaca dengan benar oleh backend	PASS
Access log dan error log tersimpan di direktori host	PASS
Hanya proxy yang memiliki published port; backend tidak terekspos	PASS
Mahasiswa mampu menjelaskan alur request client–proxy–backend	PASS

6. Evaluasi dan Latihan Mandiri

1. Mengapa reverse proxy tidak seharusnya menjalankan semua logic aplikasi?
2. Apa perbedaan TLS termination dan end-to-end TLS?
3. Bagaimana cara mengisolasi backend agar tidak langsung diakses dari host?
4. Apa konsekuensi menyimpan private key TLS di bind mount?
5. Bandingkan log Nginx dan log Apache dari sisi format dan kegunaan debugging.

6.1 Jawaban

1. Reverse proxy = lapisan kebijakan, bukan lapisan bisnis. TLS termination, routing, penambahan header, pembatasan ukuran request, dan pencatatan log cukup dikelola di satu tempat sehingga mudah diaudit dan tidak tersebar per aplikasi. Aplikasi tetap memegang konteks domain seperti autentikasi, otorisasi, dan validasi input; bila logika bisnis dipaksakan ke proxy, setiap perubahan aplikasi ikut menyentuh konfigurasi infrastruktur dan saling mengikat satu sama lain. Pemisahan ini juga membuat backend dapat diskalakan tanpa mengubah perilaku proxy.
2. **TLS termination** menghentikan enkripsi di proxy: client melakukan TLS ke proxy, proxy mendekripsi lalu meneruskan HTTP biasa (atau TLS baru) ke backend. Client hanya mempercayai sertifikat yang trusting ke CA proxy, private key hanya ada di proxy, tetapi proxy menjadi titik kepercayaan tambahan dan data intra-network tidak terenkripsi. **End-to-end TLS** mempertahankan enkripsi dari client sampai backend; proxy hanya meneruskan byte dan tidak dapat membaca isi request, namun backend memerlukan trust store dan konfigurasi sertifikat sendiri, serta penggunaan CPU lebih tinggi di setiap hop.
3. Backend tidak diberi ports sama sekali (cukup expose agar dapat dijangkau service lain dalam network Compose), published port proxy dibatasi ke loopback 127.0.0.1, dan backend ditempatkan pada network internal terpisah dari proxy. Backend juga ditambahkan allowlist sumber X-Forwarded-* sehingga header dari sumber yang bukan proxy diabaikan. Semua jalur trafik eksternal dipaksa melewati proxy, sehingga klaim forwarded header dapat dibuktikan lewat topologi, bukan sekadar konfigurasi aplikasi.
4. Private key pada bind mount berarti file rahasia berada di host dalam bentuk plaintext. Risiko berasal dari backup, image, cache build, atau git add yang tidak disengaja. Jika bocor, penyerang dapat melakukan impersonation dan mendelegitimasi kepercayaan server terhadap seluruh client, bukan hanya satu endpoint. Mitigasi laboratorium berupa permission 600 dan mount read-only; pada produksi key sebaiknya datang dari secret manager atau Docker secret dengan rotasi terjadwal.
5. Nginx memakai satu format combined yang memuat alamat client, request, status, ukuran body, referrer, dan user agent, serta dapat ditambah \$request_time dan \$upstream_response_time untuk melihat latency proxy–upstream. Apache menulis log melalui mod_log_config dengan format common / combined yang setara, ditambah %D untuk waktu handler, dan mendukung pencatatan per virtual host. Kegunaan debugging pada lab: log Nginx dipakai untuk menelusuri routing, redirect, dan penerusan header di sisi proxy, sedangkan log Apache dipakai memastikan request benar-benar sampai ke origin. Keduanya sama-sama tidak menyimpan request body, sehingga debugging POST tetap memerlukan access log aplikasi.

7. Analisis Hasil

7.1 Masalah yang Muncul dan Cara Mendiagnosisnya

Gejala yang Muncul	Langkah Diagnosis	Akar Penyebab
Request ke /api dilayani 503 Service Unavailable	docker compose ps menunjukkan flask-app running, lalu docker compose logs flask-app dan curl endpoint /health dari dalam container	Proses sudah hidup tetapi belum siap menerima trafik; healthcheck /health belum lulus sehingga depends_on: service_healthy menahan proxy
curl -k https://localhost:8443 dan openssl s_client menampilkan	Menjalankan openssl s_client -connect localhost:8443 -servername localhost	Sertifikat ditandatangani sendiri sehingga tidak ada

Gejala yang Muncul	Langkah Diagnosis	Akar Penyebab
Verification error: self-signed certificate	untuk melihat verifikasi, masa berlaku, dan SAN	rantai kepercayaan ke CA yang dipercaya
Header X-Forwarded-Proto tidak dapat dipercaya jika backend dapat diakses langsung	docker compose ps dan docker network inspect untuk memastikan hanya proxy yang punya <i>published port</i>	Trust header bergantung pada topologi, bukan pada aplikasi saja
Port 8080 dan 8443 gagal dipakai ulang pada sesi berikutnya	ss -lntp untuk melihat proses yang masih memegang port	Container tidak dihentikan, atau proses lain memakai port yang sama

7.2 Risiko Keamanan dan Operasional pada Bab Ini

- **Sertifikat self-signed.** Browser dan client tidak mempercayai sertifikat laboratorium sehingga rentan terhadap *impersonation* bila dibawa ke produksi. Nilai Verification error: self-signed certificate pada openssl s_client adalah pengingat bahwa proteksi yang diperoleh baru sebatas enkripsi, belum termasuk autentikasi identitas server.
- **Private key pada bind mount.** certs/lab.key berada di direktori host dengan permission 600. Bila permission tidak dibatasi atau direktori ikut dikomit, private key dapat bocor. Pada produksi, key sebaiknya dipasang sebagai *secret* dengan permission minimum dan rotasi teruji.
- **TLS offloading.** Komunikasi proxy–backend memakai HTTP biasa, sehingga pada topologi multi-host data tidak terenkripsi di jalur internal. Segmentasi network menjadi kompensasi wajib.
- **Boundary tipis.** Ketiga service berada pada satu network web-net. Bila proxy dibobol, penyerang memperoleh jalur langsung ke Apache dan Flask. Rekomendasi produksi adalah memisahkan network frontend dan backend.
- **Header tepercaya.** X-Forwarded-For dan X-Forwarded-Proto dapat dipalsukan bila backend dapat diakses langsung. Pada lab ini risiko tersebut diminimalkan dengan tidak adanya published port pada backend, namun perubahan konfigurasi sekecil apa pun (misalnya menambah ports untuk debug) akan membatalkannya.
- **Log tanpa sanitasi dan retensi.** Access log mencatat path apa adanya. Path yang berisi data sensitif (token pada query string) akan tersimpan apa adanya. Selain itu, log tanpa retensi yang jelas menyulitkan audit.
- **Sertifikat kedaluwarsa tanpa otomatisasi.** -days 365 berarti layanan akan gagal dilayani atau memicu peringatan browser setelah satu tahun tanpa ada mekanisme pembaruan.
- **Waktu lokal vs UTC.** Bila log tidak dipaksa ke UTC, rekonstruksi insiden lintas lokasi menjadi ambigu karena selisih zona waktu.

7.3 Rekomendasi untuk Environment Production-like

1. **Gunakan CA terpercaya dengan renewal otomatis.** Ganti sertifikat self-signed dengan sertifikat dari CA (misalnya Let's Encrypt) dan otomatisasi renewal di reverse proxy. Simpan private key sebagai *secret* dengan permission minimum serta uji rotasi tanpa membangun ulang aplikasi.
2. **Segmentasi network.** Pisahkan network frontend (proxy) dan backend (Apache/Flask). Jangan publikasikan backend ke host dalam kondisi apa pun.
3. **Batasi binding port.** Batasi published port ke alamat manajemen atau izinkan hanya proxy ingress yang terekspos, seperti pola 127.0.0.1:8080:80 pada lab ini.
4. **Terapkan TLS modern.** Tetapkan ssl_protocols minimum TLS 1.2 (atau 1.3), modern cipher suite, HSTS setelah domain tervalidasi, dan redirect HTTP→HTTPS yang konsisten.
5. **Validasi forwarded header hanya dari proxy terpercaya.** Backend harus menolak X-Forwarded-* yang berasal dari sumber selain proxy; tambahkan allowlist IP proxy pada aplikasi.
6. **Terapkan liveness dan readiness check.** Liveness untuk memastikan proses hidup, readiness untuk memastikan layanan siap menerima trafik, dengan depends_on: condition: service_healthy agar proxy tidak meneruskan trafik ke backend yang belum siap.
7. **Perlakukan konfigurasi proxy sebagai kode.** Terapkan review, uji sintaks (nginx -t), test integrasi, pencatatan versi, dan rollback yang sudah teruji.
8. **Sanitasi dan rotasi log.** Hilangkan data sensitif dari log, gunakan timestamp UTC, dan tetapkan kebijakan retensi yang jelas.
9. **Terapkan rate limiting dan batas ukuran request.** Tambahkan client_max_body_size untuk mencegah 413 Payload Too Large dan limit_req untuk mengurangi risiko resource exhaustion.

8. Refleksi Keamanan dan Operasional

- **Risiko keamanan:** private key pada bind mount dapat bocor; TLS self-signed tidak dipercaya client sehingga membuka kemungkinan *impersonation*; header `X-Forwarded-*` rawan *spoofing* bila backend terekspos; route longgar dapat membuka endpoint internal; semua service dalam satu network memperluas *blast radius* kompromi.
- **Risiko operasional:** sertifikat kedaluwarsa tanpa otomatisasi renewal menyebabkan outage; perubahan konfigurasi tanpa *rollback* teruji dapat mengambil seluruh layanan; log tanpa retensi dan *timestamp* konsisten melemahkan *auditability*.
- **Keterbatasan lingkungan:** konfigurasi TLS (cipher default) dapat berubah seiring versi modul atau base image, sehingga uji penerimaan perlu dijalankan ulang setelah perubahan dependency. Pada bab ini uji hanya dilakukan pada `nginx:alpine` dan `httpd:2.4-alpine` versi terkini.

9. Kesimpulan

Praktikum Bab 4 berhasil membangun web stack tiga lapis dengan Nginx sebagai reverse proxy dan TLS termination, Apache sebagai origin server, serta Flask sebagai backend API. Seluruh trafik dikendalikan melalui Nginx dengan routing, forwarded headers, TLS, dan healthcheck.

Eksperimen juga membuktikan pentingnya segmentasi network dan pembatasan exposure port untuk menjaga keamanan backend. Selain itu, penggunaan konfigurasi sebagai kode, pengujian, dan evidence log menjadi dasar pengelolaan web layer yang akan diterapkan pada praktikum database service berikutnya.